

Автоматизация UI
тестирования.

Selenium

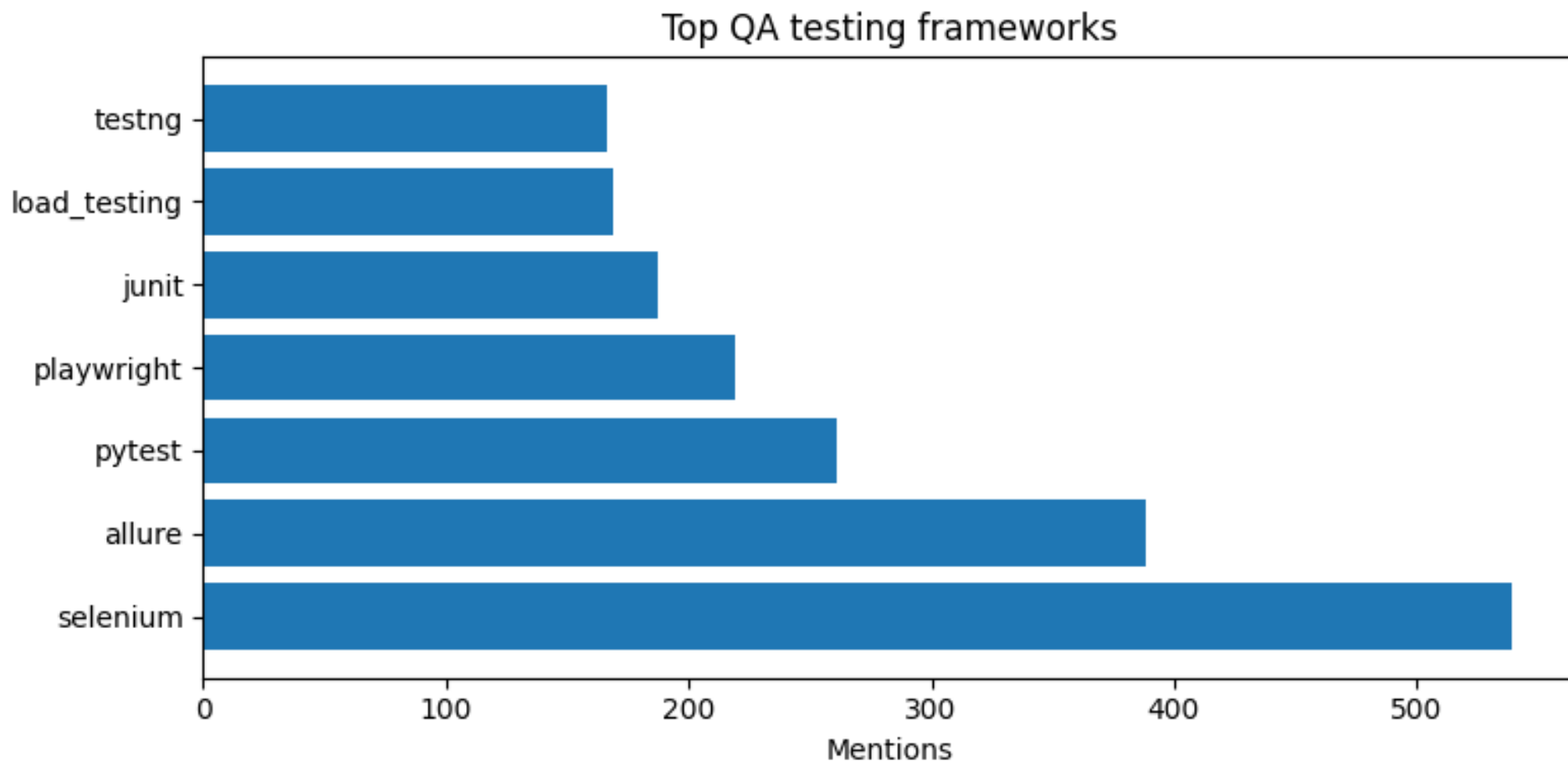
Инструменты для автоматизации UI- тестирования веб-приложений

Инструмент	Языки программирования	Поддерживаемые браузеры	Особенности
Selenium	Java, Python, C# JavaScript (Node.js), Ruby	Chrome, Firefox, Safari	Широкие возможности, поддержка различных языков
Cypress	JavaScript	Chrome, Firefox	Высокая скорость, удобный интерфейс
TestCafe	JavaScript, TypeScript	Все основные браузеры	Не требует плагинов, удобные средства отладки
Puppeteer	JavaScript, TypeScript	Chrome, Chromium	Высокоуровневый API, создание скриншотов
Playwright	JavaScript, TypeScript, Python	Chrome, Firefox, Safari	Поддержка нескольких браузеров, мощные средства отладки

Тренды:

- Playwright набирает популярность из-за скорости и удобства.
- Cypress доминирует во frontend-тестировании.
- Appium + Selenium остаются стандартом для мобильных и кросс-браузерных тестов.

На графике показано распределение упоминаний тестовых фреймворков и инструментов автоматизации в QA-вакансиях за 2025 год.



<https://habr.com/ru/articles/982052/>



Selenium

<https://www.selenium.dev/>

Selenium автоматизирует браузеры. Вот и все!

Что вы сделаете с этой силой, зависит только от вас.



Selenium WebDriver



Selenium IDE



Selenium Grid

В 2018 году **WebDriver** стал рекомендацией **World Wide Web Consortium (W3C)**. Что это значит? Основные поставщики браузеров (Mozilla, Google, Apple, Microsoft) поддерживают WebDriver и постоянно работают над улучшением браузеров и кода управления браузерами, что приводит к более единообразному поведению в разных браузерах, делая ваши скрипты автоматизации более стабильными.

Немного истории

Selenium был создан Джейсоном Хаггинсом в 2004 году. Инженер ThoughtWorks, он работал над веб-приложением, которое требовало частого тестирования. Он создал программу на JavaScript «JavaScriptTestRunner», которая автоматически контролировала действия браузера. Это было началом....

Далее Selenium активно развивался стараниями разных разработчиков.



Название «Selenium» (в переводе с англ. — селен) стало использоваться после того, как в одном из своих электронных писем **Хаггинс** пошутил о конкурирующем проекте, имеющем название «Mercury Interactive QuickTest Professional» (в переводе с англ. — ртуть), написав о том, что можно вылечиться от отравления ртутью, принимая с пищей селен.

Selenium Webdriver

Простое кроссбраузерное тестирование

Firefox, Chrome, IE, Opera, Safari и т.д.

Поддерживает множество языков программирования

Java, C#, PHP, Ruby, Perl, Python и т.д.

Есть фреймворки, облегчающие разработку

Data Driven, Keyword driven & Hybrid

Параллельное тестирование

Несколько браузеров одновременно

Хорошая статья про Selenium <https://smartiga.ru/blog/selenium-webdriver-basics>

Основными понятиями в Selenium Webdriver являются:

- ✓ **Webdriver** - самая важная сущность, ответственная за управление браузером. Основной ход скрипта строится именно вокруг экземпляра этой сущности.
- ✓ **Webelement** - вторая важная сущность, представляющая собой абстракцию над веб-элементом (кнопки, ссылки, поля ввода и др.). Webelement инкапсулирует методы для взаимодействия пользователя с элементами и получения их текущего статуса.
- ✓ **By** - абстракция над локатором веб-элемента. Этот класс инкапсулирует информацию о селекторе(например, CSS), а также сам локатор элемента, то есть всю информацию, необходимую для нахождения нужного элемента на странице.

1 WEBDRIVER

2 WEBELEMENT

3 BY

```
public void simpleScenario() {
```

```
    WebDriver driver = new ChromeDriver();  
    driver.get("http://www.google.com/");
```

```
    By searchBtnSelector = By.xpath("//input[@name='btnK']");
```

```
    WebElement searchButton = driver.findElement(searchBtnSelector);  
    searchButton.click();
```

```
    driver.quit();
```

```
}
```

Working with browser

Looking for elements

Working with elements

Java

Python

Для поиска элементов доступно два метода:

1. Первый - найдёт только первый элемент, удовлетворяющий локатору:

```
WebElement element = driver.findElement(By.id("#element_id"));
```

```
elem = driver.find_element(By.ID, "element_name")
```

2. Второй - вернёт весь список элементов, удовлетворяющих запросу:

```
List elements = driver.findElements(By.name("elements_name"))
```

```
elems = driver.find_elements(By.ID, "element_name")
```

Типы локаторов

Поскольку Webdriver - это инструмент для автоматизации веб приложений, то большая часть работы с ним это работа с веб элементами. **WebElements** - ни что иное, как **DOM объекты, находящиеся на веб странице**. А для того, чтобы осуществлять какие-то действия над DOM объектами / веб элементами необходимо их точным образом определить(найти).

```
WebElement element = driver.findElement(By.<Selector>);
```

Таким образом в Webdriver определяется нужный элемент.

By - класс, содержащий статические методы для идентификации элементов.

Java

Python

1. By.id

```
WebElement element = driver.findElement(By.id("element_id"));
```

```
elem = driver.find_element(By.ID, "element_name")
```

2. By.name

```
WebElement element = driver.findElement(By.name("element_name"));
```

```
elem = driver.find_element(By.NAME, "element_name")
```

3. By.className

```
WebElement element = driver.findElement(By.className("element_class"));
```

```
elem = driver.find_element(By.CLASS_NAME, "element_name")
```

4. By.TagName

```
List<WebElement> elements = driver.findElements(By.tagName("a"));
```

5. By.LinkText

```
WebElement element = driver.findElement(By.linkText("text"));
```

6. By.PartialLinkText

```
WebElement element = driver.findElements(By.partialLinkText("text"));
```

7. By.cssSelector

Java

Python

```
<div class='main'>
  <p>text</p>
  <p>Another text</p>
</div>
```

```
WebElement element=driver.FindElement(By.cssSelector("div.main"));
elem = driver.find_element(By.CSS_SELECTOR, "div.main")
```

8. By.XPath

```
<div class='main'>
  <p>text</p>
  <p>Another text</p>
</div>
```

```
WebElement element = driver.findElement(By.xpath("//div[@class='main']"));
elem = driver.find_element(By.XPATH, «//div[@class='main']»)
```

Возможность	CSS Селекторы	XPath	⬇
Выбор по атрибутам	✅ Да (<code>[type='text']</code>)	✅ Да (<code>[@type='text']</code>)	
Выбор по классу/ID	✅ Да (<code>.class</code> , <code>#id</code>)	✅ Да	
Иерархия (Родитель -> Потомок)	✅ Да (<code>div > span</code>)	✅ Да (<code>/</code> или <code>//</code>)	
Иерархия (Потомок -> Родитель)	❌ Нет	✅ Да (<code>/..</code> или <code>parent::</code>)	
Поиск по тексту элемента	❌ Нет (стандартный CSS не умеет)	✅ Да (<code>text()='Войти'</code>)	
Логические операторы (AND, OR)	❌ Ограничено	✅ Да (<code>and</code> , <code>or</code>)	
Выбор по индексу	❌ Нет (нужен <code>:nth-child</code>)	✅ Да (<code>[1]</code> , <code>[last()]</code>)	
Относительная навигация	❌ Нет	✅ Да (можно идти от найденного элемента в любую сторону)	

Рекомендации: Что выбирать?

Приоритет **CSS**:

- ✓ Используйте CSS-селекторы по умолчанию в 90% случаев.
- ✓ Они быстрее, чище и легче поддерживаются.
- ✓ Идеальны для поиска по id, class, стандартным атрибутам.

Используйте **XPath**, когда:

- ✓ Нужно найти элемент по видимому тексту (и нет других уникальных атрибутов).
- ✓ Нужно найти родительский элемент относительно дочернего.
- ✓ Сложная логика выбора (например, "кнопка, у которой есть атрибут X ИЛИ атрибут Y").
- ✓ CSS-селектор не может выразить нужную зависимость в DOM-дереве.

Чего избегать:

- ✓ Избегайте абсолютных путей в XPath (начинающихся с /html/...).
- ✓ Избегайте поиска по тексту в XPath, если текст может измениться (лучше попросить разработчиков добавить data-testid или уникальный класс).
- ✓ Избегайте слишком сложных селекторов (лучше добавить уникальный атрибут в код приложения, чем писать монструозный локатор).

Итог

CSS — это выбор для скорости и простоты. XPath — это "тяжелая артиллерия" для сложных случаев, когда возможностей CSS недостаточно. Хороший инженер по автотестам владеет обоими инструментами и знает, когда переключиться с одного на другой.

Селекторы бывают 2-х видов



- **Ненормальные** (правой кнопкой мыши - copy selector)

```
body > div.wrapper > div.profile > div > div >  
main > div.profile-game-headline >  
div.mod-static.profile-game-select >  
div.profile-game-select-value.js-select-current-  
item > span > span.profile-game-select-title
```

- **Все остальные**

Пример

```
▼ <body>
  ▶ <header>...</header>
  ▼ <main role="main">
    ▶ <section class="jumbotron text-center">...</section>
    ▼ <div class="album py-5 bg-light">
      ▼ <div class="container">
        ▼ <div class="row"> flex
          ▼ <div class="col-sm-4">
            ▶ <div class="card mb-4 box-shadow">...</div> flex
            </div>
          ▼ <div class="col-sm-4">
            ▼ <div class="card mb-4 box-shadow"> flex
              .. →  == $0
              ▶ <div class="card-body">...</div>
              </div>
            </div>
          </div>
        </div>
      </div>
    </main>
  </body>
```

Селекторы, которые генерирует браузер по кнопке "скопировать css селектор" или расширения, зачастую строят полный путь, начиная от body — а это очень нестабильный селектор, писать такие в своем коде это плохая практика.

При малейшем изменении структуры страницы все ваши селекторы потеряют актуальность.

Сгенерированный

body > main > div > div > div > div:nth-child(2) > div > img

Написанный вручную

div.col-sm-4:nth-child(2) img

Взаимодействие с веб-элементами

Над элементом можно выполнить только 5 основных команд:

- click (applies to any element)
- send keys (only applies to text fields and content editable elements)
- clear (only applies to text fields and content editable elements)
- submit (only applies to form elements)
- select (see Select List Elements)

```
# Отметить checkbox "I'm the robot".  
option1 = driver.find_element(By.CSS_SELECTOR, value: "[for='robotCheckbox']")  
option1.click()
```

Действия с клавиатурой

https://www.selenium.dev/documentation/webdriver/actions_api/keyboard/

Функция `send_keys()` принимает различные ключи в качестве параметра. Поэтому необходимо импортировать ключи перед использованием этой функции.

```
1  from selenium import webdriver
2  from selenium.webdriver.common.keys import Keys
3  from selenium.webdriver.common.by import By
4
5  driver = webdriver.Chrome()
6  driver.get("http://www.python.org")
7  elem = driver.find_element(By.NAME, value: "q")
8  elem.clear()
9  elem.send_keys("pycon")
10 elem.send_keys(Keys.RETURN)
11 assert "No results found." not in driver.page_source
12 driver.close()
```

Ввод в текстовое поле

Нажатие клавиши Ввод

Клавиша Return?

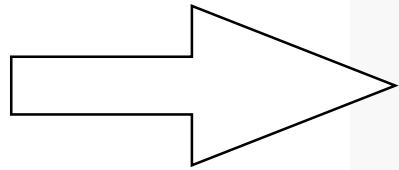


Клавиши *Return* (красный) и *Enter*  (зелёный) на буквенной и цифровой клавиатурах, соответственно

Действия с клавиатурой

С помощью клавиатуры можно выполнить только 2 действия: **нажатие клавиши** и **отпускание нажатой клавиши**. Помимо поддержки символов ASCII, каждая клавиша клавиатуры имеет представление, которое можно нажимать или отпускать в определенных последовательностях.

В Selenium можно выполнять все операции с клавиатурой. Класс `selenium.webdriver.common.keys` содержит различные методы, которые можно использовать для этой цели.



```
ActionChains(driver) \
    .key_down(Keys.SHIFT) \
    .send_keys("a") \
    .key_up(Keys.SHIFT) \
    .send_keys("b") \
    .perform()
```

ActionChains — это способ автоматизации низкоуровневых взаимодействий, таких как движения мыши, действия кнопок мыши, нажатия клавиш и взаимодействия с контекстным меню. Это полезно для выполнения более сложных действий, таких как наведение и перетаскивание.

Генерация действий пользователя.

Когда вы вызываете методы для действий на объекте ActionChains, действия сохраняются в очереди в объекте ActionChains. Когда вы вызываете perform(), события запускаются в том порядке, в котором они поставлены в очередь.

ActionChains можно использовать в цепочке:

```
ActionChains(driver).move_to_element(menu).click(hidden_submenu).perform()
```

Или действия можно выстроить в очередь одно за другим, а затем выполнить:

```
actions = ActionChains(driver)
actions.move_to_element(menu)
actions.click(hidden_submenu)
actions.perform()
```

https://www.selenium.dev/selenium/docs/api/py/webdriver/selenium.webdriver.common.action_chains.html

```
from selenium import webdriver
from selenium.webdriver.common.keys import Keys
from selenium.webdriver.common.action_chains import ActionChains
```

```
driver = webdriver.Firefox(executable_path="C:\\geckodriver.exe")
driver.get("url")
```

```
actions = ActionChains(driver)
actions.send_keys(value=username)
actions.send_keys(keys.TAB)
actions.send_keys(value=password)
actions.send_keys(keys.ENTER)
actions.perform()
driver.quit()
```

Действия с мышью

https://www.selenium.dev/documentation/webdriver/actions_api/mouse/

С помощью мыши можно выполнить только 3 действия: нажать кнопку, отпустить нажатую кнопку и переместить мышь.

```
clickable = driver.find_element(By.ID, "click")
ActionChains(driver)\
    .click(clickable)\
    .perform()
```

```
clickable = driver.find_element(By.ID, "clickable")
ActionChains(driver)\
    .click_and_hold(clickable)\
    .perform()
```

Действия с мышью

Щелчок правой кнопкой.

```
clickable = driver.find_element(By.ID, "clickable")
ActionChains(driver)\
    .context_click(clickable)\
    .perform()
```

Двойной щелчок

```
clickable = driver.find_element(By.ID, "clickable")
ActionChains(driver)\
    .double_click(clickable)\
    .perform()
```


Действия с мышью

Перейти к элементу.

```
hoverable = driver.find_element(By.ID, "hover")
ActionChains(driver)\
    .move_to_element(hoverable)\
    .perform()
```

Перетащить элемент

```
draggable = driver.find_element(By.ID, "draggable")
droppable = driver.find_element(By.ID, "droppable")
ActionChains(driver)\
    .drag_and_drop(draggable, droppable)\
    .perform()
```

Настройка ожиданий

Ожидания - неперенный атрибут любых UI тестов для динамических приложений. Нужны они для синхронизации работы UI и тестового скрипта.

Скрипт выполняется намного быстрее реакции приложения на команды, поэтому часто в скриптах необходимо дожидаться определенного состояния приложения для дальнейшего с ним взаимодействия.

Самый простой пример - переход по ссылке и нажатие кнопки:

```
driver.get("http://google.com")  
driver. find_element(By.ID("element_id")).click();
```

В данном случае необходимо дождаться пока не появится кнопка с id = element_id и только потом совершать действия над ней. Для этого и существуют ожидания.



Для ожиданий можно использовать библиотеку **time** в Python.

```
from selenium import webdriver
from selenium.webdriver.common.by import By
import time

browser = webdriver.Chrome()
browser.get("http://suninjuly.github.io/wait1.html")

time.sleep(1)

button = browser.find_element(By.ID, "verify")
button.click()
message = browser.find_element(By.ID, "verify_message")

assert "successful" in message.text
```

Вывод: решение с `time.sleep()` плохое: оно не масштабируемое и трудно поддерживаемое.

- А если другие элементы тоже будут появляться с задержкой?
- А если изменится время задержки?
- А еще на разных машинах с разной скоростью интернета элементы могут появляться через разные промежутки времени.

Неявные ожидания (Implicit Waits)

Неявные ожидания конфигурируют экземпляр WebDriver делать многократные попытки найти элемент (элементы) на странице в течение заданного периода времени, если элемент не найден сразу.

```
from selenium import webdriver
from selenium.webdriver.common.by import By

browser = webdriver.Chrome()
# говорим WebDriver искать каждый элемент в течение 5 секунд
browser.implicitly_wait(5)

browser.get("http://suninjuly.github.io/wait1.html")

button = browser.find_element(By.ID, "verify")
button.click()
message = browser.find_element(By.ID, "verify_message")

assert "successful" in message.text
```

В данном примере наличие элемента будет проверяться каждые 500 мс. Как только элемент будет найден, мы сразу перейдем к следующему шагу в тесте.

Явные ожидания (Explicit Waits)

Методы `find_element` проверяют только то, что элемент появился на странице. В то же время элемент может иметь дополнительные свойства, которые могут быть важны для наших тестов. Рассмотрим пример с кнопкой, которая отправляет данные:

- ✓ Кнопка может быть неактивной, то есть её нельзя кликнуть;
- ✓ Кнопка может содержать текст, который меняется в зависимости от действий пользователя. Например, текст "Отправить" после нажатия поменяется на "Отправлено";
- ✓ Кнопка может быть перекрыта каким-то другим элементом или быть невидимой.

***Пример:** Если мы хотим в тесте кликнуть на кнопку, а она в этот момент неактивна, то WebDriver все равно проэмулирует действие нажатия на кнопку, но данные не будут отправлены. Чтобы тест был надежным, нужно не только найти кнопку на странице, но и дождаться, когда кнопка станет кликабельной.*

Для реализации подобных ожиданий в Selenium WebDriver существует понятие **явных ожиданий (Explicit Waits)**, которые позволяют задать специальное ожидание для конкретного элемента.

Задание явных ожиданий реализуется с помощью инструментов WebDriverWait и expected_conditions.

```
from selenium.webdriver.common.by import By
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC
from selenium import webdriver

browser = webdriver.Chrome()

browser.get("http://suninjuly.github.io/wait2.html")

# говорим Selenium проверять в течение 5 секунд, пока кнопка не станет кликабельной
button = WebDriverWait(browser, 5).until(
    EC.element_to_be_clickable((By.ID, "verify"))
)
button.click()
message = browser.find_element(By.ID, "verify_message")

assert "successful" in message.text
```

```
# говорим Selenium проверять в течение 5 секунд, пока кнопка не станет кликабельной
button = WebDriverWait(browser, 5).until(
    EC.element_to_be_clickable(By.ID, "verify"))
button.click()
```

Условие ожидания

В модуле `expected_conditions` есть много других правил, которые позволяют реализовать необходимые ожидания:

- `title_is`
- `title_contains`
- `visibility_of_element_located`
- `visibility_of`
- `presence_of_all_elements_located`
- `text_to_be_present_in_element`
- `element_to_be_clickable`
- `element_to_be_selected`
- `element_located_to_be_selected`
- `element_selection_state_to_be`
- `element_located_selection_state_to_be`
- `alert_is_present ...`

Описание каждого правила можно найти на [сайте](#).

Шпаргалка по Expected Conditions в Selenium

1. title_is : Ожидает, что **заголовок страницы будет точно соответствовать заданному тексту.**

```
wait.until(EC.title_is('Expected Title'))
```

2. title_contains : Ожидает, что **заголовок страницы будет содержать заданный текст.**

```
wait.until(EC.title_contains('Expected Text'))
```

3. url_to_be : Ожидает, что **URL страницы будет точно соответствовать заданному URL.**

```
wait.until(EC.url_to_be('https://example.com'))
```

4. url_contains : Ожидает, что **URL страницы будет содержать заданный текст.**

```
wait.until(EC.url_contains('example'))
```

5. presence_of_element_located : Ожидает, что **элемент будет присутствовать в DOM.**

```
wait.until(EC.presence_of_element_located((By.ID, 'element_id')))
```

6. visibility_of_element_located : Ожидает, что **элемент станет видимым.**

```
wait.until(EC.visibility_of_element_located((By.XPATH, 'xpath_expression')))
```

7. visibility_of : Ожидает, что **элемент станет видимым и отображаемым на странице.**

```
element = wait.until(EC.visibility_of(driver.find_element(By.ID, 'element_id')))
```


8. invisibility_of_element_located : Ожидает, что **элемент станет невидимым**.

```
wait.until(EC.invisibility_of_element_located((By.CSS_SELECTOR, 'css_selector')))
```

9. text_to_be_present_in_element : Ожидает, что **определенный текст появится в элементе**.

```
wait.until(EC.text_to_be_present_in_element((By.ID, 'element_id'), 'expected
```

10. element_to_be_clickable : Ожидает, что **элемент станет кликабельным**.

```
wait.until(EC.element_to_be_clickable((By.ID, 'element_id')))
```

11. element_to_be_selected : Ожидает, что **элемент будет выбран**.

```
wait.until(EC.element_to_be_selected((By.ID, 'element_id')))
```

12. element_selection_state_to_be : Ожидает, что **состояние выбора элемента будет соответствовать заданному состоянию (True/False)**.

```
wait.until(EC.element_selection_state_to_be((By.ID, 'element_id'), True))
```

13. element_located_selection_state_to_be : **Ожидает, что состояние выбора элемента (по его локатору) будет соответствовать заданному состоянию (True/False)**.

```
wait.until(EC.element_located_selection_state_to_be((By.ID, 'element_id'), True))
```

14. alert_is_present : Ожидает, что **появится всплывающее окно (alert)**.

```
wait.until(EC.alert_is_present())
```

Exceptions

Если при поиске элемента произойдет ошибка, то WebDriver выбросит одно из следующих исключений:

- ✓ **NoSuchElementException** - если элемент не был найден за отведенное время.
- ✓ **StaleElementReferenceException** - если элемент был найден в момент поиска, но при последующем обращении к элементу DOM изменился. Например, мы нашли элемент Кнопка и через какое-то время решили выполнить с ним уже известный нам метод click. Если кнопка за это время была скрыта скриптом, то метод применять уже бесполезно — элемент "устарел" (stale) и мы увидим исключение.
- ✓ **ElementNotVisibleException** - если элемент был найден в момент поиска, но сам элемент невидим (например, имеет нулевые размеры), и реальный пользователь не смог бы с ним взаимодействовать.

Знание причин появления исключений помогает отлаживать тесты и понимать, где находится баг в случае его возникновения.

Проверка наличия элементов в Selenium

<https://qarocks.ru/elements-presence-verifying-in-selenium/>

Проверка наличия элементов страницы в Selenium чаще всего осуществляется при помощи трех команд:

- **verifyElementPresent / verifyElementNotPresent** – для проверки наличия или отсутствия элемента
- **verifyTextPresent / verifyTextNotPresent** – для проверки наличия или отсутствия текста
- **verifyElementPositionLeft / verifyElementPositionTop** – для проверки местоположения элемента на странице

Настройка параметров Selenium WebDriver

1. Импорт опций Chrome

```
from selenium import webdriver  
from selenium.webdriver.chrome.options import Options
```

2. Инициализация опций Chrome

```
chrome_options = Options()
```

3. Добавление желаемых возможностей

```
chrome_options.add_argument("--disable-extensions")
```

4. Добавление желаемых возможностей сессии

```
driver = webdriver.Chrome(chrome_options=chrome_options)
```

Список доступных и наиболее часто используемых аргументов для класса ChromeOptions:

- **start-maximized**: открывает Chrome в режиме максимизации.
- **incognito**: открывает Chrome в режиме инкогнито.
- **headless**: открывает Chrome в безголовом режиме.
- **disable-extensions**: отключает существующие расширения в браузере Chrome
- **disable-popup-blocking**: отключает всплывающие окна, отображаемые в браузере Chrome
- **make-default-browser**: делает Chrome браузером по умолчанию
- **version**: выводит версию браузера Chrome
- **disable-infobars**: запрещает Chrome отображать уведомление «Chrome управляется автоматическим программным обеспечением».

УПРАВЛЕНИЕ WEBDRIVER-МОДОМ И USER-AGENT

Как притворится человеком

https://www.youtube.com/watch?v=bg_0nB4-DmM

Test Name	Result
User Agent (Old)	Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/130.0.0.0 Safari/537.36
WebDriver (New)	missing (passed)
Chrome (New)	present (passed)
Permissions (New)	prompt
Plugins Length (Old)	5
Languages (Old)	ru-RU,ru,en-US,en

Сайт обнаруживает
автотест

Test Name	Result
User Agent (Old)	Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/114.0.0.0 Safari/537.36
WebDriver (New)	present (failed)
Chrome (New)	present (passed)
Permissions (New)	prompt
Plugins Length (Old)	5
Languages (Old)	en-GB,en-US,en

Этот сайт демонстрирует определение человека или робота

<https://intoli.com/blog/not-possible-to-block-chrome-headless/chrome-headless-test.html>

статья об этом <https://intoli.com/blog/not-possible-to-block-chrome-headless/>

```
import time
from selenium import webdriver
from webdriver_manager.chrome import ChromeDriverManager
from selenium.webdriver.chrome.options import Options
from selenium.webdriver.chrome.service import Service
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC
```

Можно отключить
видимость сайтом
использование
webdriver

```
options = Options()
options.add_argument("--window-size=1920,1080")
options.add_argument("--disable-blink-features=AutomationControlled")
```

```
service = Service(ChromeDriverManager().install())
driver = webdriver.Chrome(service=service, options=options)
wait = WebDriverWait(driver, 5, poll_frequency=1)
```

```
driver.get("https://intoli.com/blog/not-possible-to-block-chrome-headless/chrome-headless")

time.sleep(3)
```

Работа с User-Agent

Selenium использует строку User-Agent для идентификации себя при выполнении HTTP-запросов.

Можно статически изменить User-Agent

```
custom_user_agent = "Mozilla/5.0 (Windows NT 10.0; Win64; x64)  
AppleWebKit/537.36 (KHTML, как Gecko) Chrome/122.0.0.0 Safari/537.36"
```

```
chrome_options = webdriver.ChromeOptions()  
chrome_options.add_argument(f'--user-agent={custom_user_agent}')
```



```
import time
from selenium import webdriver
from fake_useragent import UserAgent
```

```
ua = UserAgent()
chrome_options = webdriver.ChromeOptions()
chrome_options.add_argument(f'--user-agent={ua.random}')
```

```
driver = webdriver.Chrome(options=chrome_options)
driver.get("https://httpbin.org/user-agent")
```

```
time.sleep(10)
driver.quit()
```

Фикстуры (Fixtures)

Фикстуры в контексте PyTest — это вспомогательные функции для тестов, которые не являются частью тестового сценария.

Фикстуры можно использовать для самых разных целей:

- **подготовки тестового окружения** и очистка тестового окружения и данных после завершения теста;
- **для подключения к базе данных**, с которой работают тесты;
- **для повторного использования кода**, фикстуры позволяют разработчикам переиспользовать код настройки и очистки для множества тестов, уменьшая дублирование и упрощая изменения.;
- **подготовки данных в текущем окружении** с помощью API-методов и т.д.
- они **обеспечивают изоляцию тестов**, гарантируя, что каждый тест начинается с одного и того же известного состояния, что делает тесты более предсказуемыми и надежными.

Пример фикстуры в Pytest

```
import pytest

# Простая фикстура для подготовки данных
@pytest.fixture
def some_data():
    return [1, 2, 3, 4, 5]

# Пример использования фикстуры
def test_fixture(some_data):
    assert len(some_data) == 5
    assert sum(some_data) == 15
```

```
from selenium import webdriver
from selenium.webdriver.common.by
import By link = "http://selenium1py.pythonanywhere.com/"
```

```
class TestMainPage1():
```

```
    @classmethod
    def setup_class(self):
        print("\nstart browser for test suite..")
        self.browser = webdriver.Chrome()
```

```
    @classmethod
    def teardown_class(self):
        print("quit browser for test suite..")
        self.browser.quit()
```

```
    def test_guest_should_see_login_link(self):
        self.browser.get(link)
        self.browser.find_element(By.CSS_SELECTOR, "#login_link")
```

```
    def test_guest_should_see_basket_link_on_the_main_page(self):
        self.browser.get(link)
        self.browser.find_element(By.CSS_SELECTOR, ".basket-mini .btn-group > a")
```

Фикстуры для подготовки и очистки тестового окружения

- **префиксы `setup_*`, `teardown_*`** отвечают за порядок исполнения фикстур: до чего-то, после чего-то.
- **постфиксы `*_class`, `*_method`** и другие отвечают за уровень применения фикстур: ко всему классу, к каждому методу в классе и тд.

Фикстуры, возвращающие значение

```
import pytest
from selenium import webdriver
from selenium.webdriver.common.by import By

link = "http://selenium1py.pythonanywhere.com/"
```

```
@pytest.fixture
def browser():
    print("\nstart browser for test..")
    browser = webdriver.Chrome()
    return browser
```

Для того, чтобы функцию зарегистрировать как фикстуру, в pytest есть специальный маркер (декоратор) **@pytest.fixture**

```
class TestMainPage1():
    # вызываем фикстуру в тесте, передав ее как параметр
    def test_guest_should_see_login_link(self, browser):
        browser.get(link)
        browser.find_element(By.CSS_SELECTOR, "#login_link")

    def test_guest_should_see_basket_link_on_the_main_page(self, browser):
        browser.get(link)
        browser.find_element(By.CSS_SELECTOR, ".basket-mini .btn-group > a")
```

```
@pytest.fixture
def browser():
    print("\nstart browser for test..")
    browser = webdriver.Chrome()
    yield browser
    # ЭТОТ КОД ВЫПОЛНИТСЯ ПОСЛЕ ЗАВЕРШЕНИЯ ТЕСТА
    print("\nquit browser..")
    browser.quit()
```

setup

teardown

```
class TestMainPage1():
    # вызываем фикстуру в тесте, передав ее как параметр
    def test_guest_should_see_login_link(self, browser):
        browser.get(link)
        browser.find_element(By.CSS_SELECTOR, "#login_link")

    def test_guest_should_see_basket_link_on_the_main_page(self, browser):
        browser.get(link)
        browser.find_element(By.CSS_SELECTOR, ".basket-mini .btn-group > a")
```

Из фикстуры можно передать значение в тест-сьют с помощью оператора **yield**. При этом после yield можно добавить ещё код, который будет выполнен после кейса.

Таким образом можно сказать, что **всё, что идёт до оператора yield является “setup”, а всё, что после — “teardown”** (yield может ничего и не возвращать, а просто будет разделителем, отделяющим “setup” от “teardown”).

Область видимости scope

```
@pytest.fixture(scope="class")
def browser():
    print("\nstart browser for test..")
    browser = webdriver.Chrome()
    yield browser
    print("\nquit browser..")
    browser.quit()
```

```
class TestMainPage1():
```

```
# вызываем фикстуру в тесте, передав ее как параметр
def test_guest_should_see_login_link(self, browser):
    print("start test1")
    browser.get(link)
    browser.find_element(By.CSS_SELECTOR, "#login_link")
    print("finish test1")

def test_guest_should_see_basket_link_on_the_main_page(self, browser):
    print("start test2")
    browser.get(link)
    browser.find_element(By.CSS_SELECTOR, ".basket-mini .btn-group > a")
    print("finish test2")
```

Для фикстур можно задавать область покрытия фикстур.

Допустимые значения: **function**, **class**, **module**, **session**.

function: - это значение по умолчанию.

Автоиспользование фикстур

```
@pytest.fixture
def browser():
    print("\nstart browser for test..")
    browser = webdriver.Chrome()
    yield browser
    print("\nquit browser..")
    browser.quit()
```

`autouse=True` указывает, что фикстуру нужно запустить для каждого теста даже без явного вызова

```
@pytest.fixture(autouse=True)
def prepare_data():
    print()
    print("preparing some critical data for every test")
```

preparing some critical data for every test

start browser for test..

·
quit browser..

preparing some critical data for every test

start browser for test..

·
quit browser..

```
class TestMainPage1():
    def test_guest_should_see_login_link(self, browser):
        # не передаём как параметр фикстуру prepare_data, но она все равно выполняется
        browser.get(link)
        browser.find_element(By.CSS_SELECTOR, "#login_link")

    def test_guest_should_see_basket_link_on_the_main_page(self, browser):
        browser.get(link)
        browser.find_element(By.CSS_SELECTOR, ".basket-mini .btn-group > a")
```


Категоризация тестов

Pytest позволяет определять категории для тестов и предоставляет опции для включения или исключения категорий при запуске набора. Тест можно отметить любым количеством категорий.

```
class TestMainPage1():
```

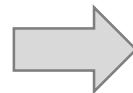
```
    @pytest.mark.smoke
```

```
    def test_guest_should_see_login_link(self, browser):  
        browser.get(link)  
        browser.find_element(By.CSS_SELECTOR, "#login_link")
```

```
    @pytest.mark.regression
```

```
    def test_guest_should_see_basket_link_on_the_main_page(self, browser):  
        browser.get(link)  
        browser.find_element(By.CSS_SELECTOR, ".basket-mini .btn-group > a")
```

В PyTest настоятельно рекомендуется регистрировать метки явно перед использованием. Для этого создается файл *pytest.ini* в корневой директории тестового проекта



```
[pytest]  
markers =  
    smoke: marker for smoke tests  
    regression: marker for regression tests
```

- ✓ Чтобы **запустить тест с нужной маркировкой**, нужно передать в командной строке параметр -m и нужную метку:

```
pytest -s -v -m smoke имя_файла.py
```

- ✓ Чтобы **запустить все тесты, не имеющие заданную маркировку**, можно использовать инверсию. Для запуска всех тестов, не отмеченных как smoke, нужно выполнить команду:

```
pytest -s -v -m "not smoke" имя_файла.py
```

- ✓ Для **запуска тестов с разными метками** можно использовать логическое ИЛИ. Запустим smoke и regression-тесты:

```
pytest -s -v -m "smoke or regression" имя_файла.py
```

- ✓ Для **запуска тестов с несколькими маркировками**, нужно использовать логическое И:

```
pytest -s -v -m "smoke and win10" имя_файла.py
```

```
@pytest.mark.smoke
@pytest.mark.win10
def test_guest_should_see_basket_
    browser.get(link)
    browser.find_element(By.CSS_SI
```

Категоризация

- Функциональные категории
- Категории по типу теста
- Категории по приоритету
- Категории по функциональности
- Кастомные категории

```
@pytest.mark.my_custom_category  
def test_custom_feature():  
    assert True
```

Функциональные

- **@pytest.mark.smoke:** Обозначает тесты для быстрой проверки базовой функциональности.
- **@pytest.mark.regression:** Обозначает тесты для проверки регрессии.
- **@pytest.mark.performance:** Тесты на производительность.
- **@pytest.mark.integration:** Тесты интеграции.
- **@pytest.mark.end_to_end:** Тесты, которые проходят через всю систему или ее большую часть.

Категории по типу теста

- `@pytest.mark.unit`: Маркировка для модульных тестов.
- `@pytest.mark.functional`: Функциональные тесты.
- `@pytest.mark.api`: Тесты для API.
- `@pytest.mark.ui`: Тесты для пользовательского интерфейса.

Категории по приоритету

- `@pytest.mark.high_priority`: Тесты высокого приоритета.
- `@pytest.mark.medium_priority`: Тесты среднего приоритета.
- `@pytest.mark.low_priority`: Тесты низкого приоритета.

Категории по функциональности

- `@pytest.mark.login`: Тесты для проверки входа в систему.
- `@pytest.mark.search`: Тесты для проверки поиска.
- `@pytest.mark.checkout`: Тесты для проверки процесса оформления заказа.

Пропуск тестов

В PyTest есть стандартные метки, которые позволяют пропустить тест при сборе тестов для запуска или запустить, но отметить особенным статусом тот тест, который ожидаемо упадёт из-за наличия бага, чтобы он не влиял на результаты прогона всех тестов.

Эти метки не требуют дополнительного объявления в pytest.ini.

```
class TestMainPage1():  
    @pytest.mark.skip  
    def test_guest_should_see_login_link(self, browser):  
        browser.get(link)  
        browser.find_element(By.CSS_SELECTOR, "#login_link")  
  
    def test_guest_should_see_basket_link_on_the_main_page(self, browser):  
        browser.get(link)  
        browser.find_element(By.CSS_SELECTOR, ".basket-mini .btn-group > a")
```

XFail: пометить тест как ожидаемо падающий

```
class TestMainPage1():

    def test_guest_should_see_login_link(self, browser):
        browser.get(link)
        browser.find_element(By.CSS_SELECTOR, "#login_link")

    def test_guest_should_see_basket_link_on_the_main_page(self, browser):
        browser.get(link)
        browser.find_element(By.CSS_SELECTOR, ".basket-mini .btn-group > a")

    @pytest.mark.xfail
    def test_guest_should_see_search_button_on_the_main_page(self, browser):
        browser.get(link)
        browser.find_element(By.CSS_SELECTOR, "button.favorite")
```

Когда баг починят, после запуска тест будет отмечен как **XPASS** (“unexpectedly passing” — неожиданно проходит).

xfail (reason...)

К маркировке **xfail** можно добавлять параметр **reason**. Чтобы увидеть это сообщение в консоли, при запуске нужно добавлять параметр **pytest -rx**.

```
@pytest.mark.xfail(reason="fixing this bug right now")
def test_guest_should_see_search_button_on_the_main_page(self, browser):
    browser.get(link)
    browser.find_element(By.CSS_SELECTOR, "button.favorite")
```

```
@pytest.mark.xfail(condition=False, *, reason=None, raises=None,
run=True, strict=xfail_strict)
```


Conftest.py — конфигурация тестов

Для хранения часто употребляемых фикстур и хранения глобальных настроек нужно использовать файл **conftest.py**, который должен лежать в директории верхнего уровня в проекте с тестами.

conftest.py:

```
import pytest
from selenium import webdriver
from selenium.webdriver.common.by import By

@pytest.fixture(scope="function")
def browser():
    print("\nstart browser for test..")
    browser = webdriver.Chrome()
    yield browser
    print("\nquit browser..")
    browser.quit()
```

test_conftest.py:

```
from selenium.webdriver.common.by import By

link = "http://selenium1py.pythonanywhere.com/"

def test_guest_should_see_login_link(browser):
    browser.get(link)
    browser.find_element(By.CSS_SELECTOR, "#login_link")
```

Параметризация тестов

PyTest позволяет запустить один и тот же тест с разными входными параметрами. Для этого используется декоратор **@pytest.mark.parametrize()**.

Пример: сайт доступен для разных языков. Напишем тест, который проверит, что для сайта с русским и английским языком будет отображаться ссылка на форму логина. Передадим в тест ссылки на русскую и английскую версию главной страницы сайта.

```
@pytest.mark.parametrize('language', ["ru", "en-gb"])
class TestLogin:
    def test_guest_should_see_login_link(self, browser, language):
        link = f"http://selenium1py.pythonanywhere.com/{language}/"
        browser.get(link)
        browser.find_element(By.CSS_SELECTOR, "#login_link")
        # этот тест запустится 2 раза

    def test_guest_should_see_navbar_element(self, browser, language):
        # этот тест тоже запустится дважды
```

Пример тестовой функции, реализующей проверку того, что определенный ввод приводит к ожидаемому выводу:

```
@pytest.mark.parametrize("test_input,expected", [("3+5", 8), ("2+4", 6), ("6*9", 42)])  
def test_eval(test_input, expected):  
    assert eval(test_input) == expected
```

Здесь декоратор определяет три разных кортежа, чтобы функция выполнялась три раза, используя их по очереди.

Проверки в тестах

Тест должен заканчиваться сравнением ожидаемого и фактического результата.

В автотестах на Python для проверки используют встроенный оператор `assert`.

python

```
1 def test_example():
2     a = 1
3     b = 2
4     assert a + b == 5
```

При несовпадении в стандартном Python будет просто `AssertionError`, и придется добавлять `print` или использовать отладчик, чтобы понять значения переменных.

В `pytest` сразу выводится полное объяснение.

Вывод в стандартном Python:

```
1 Traceback (most recent call last):
2 |   File "test.py", line 4, in <module>
3 |       assert a + b == 5
4 |   AssertionError
```

Вывод в `Pytest`:

```
1 >      assert a + b == 5
2 E      assert (1 + 2) == 5
3 E      + where 3 = (1 + 2)
```

.....

Добавление товара в корзину

```
add_to_cart_btn = wait.until(EC.element_to_be_clickable((By.ID, PRODUCT_ID)))
```

```
add_to_cart_btn.click()
```

Проверка, что счетчик корзины обновился (должно стать '1')

```
cart_badge = wait.until(EC.visibility_of_element_located((By.CLASS_NAME, CART_BADGE)))
```

```
assert cart_badge.text == "1", f"Ожидалось '1', но в корзине '{cart_badge.text}'"
```

Переход в корзину

```
cart_icon = driver.find_element(By.CLASS_NAME, CART_ICON)
```

```
cart_icon.click()
```

Проверка, что товар действительно отображается в корзине

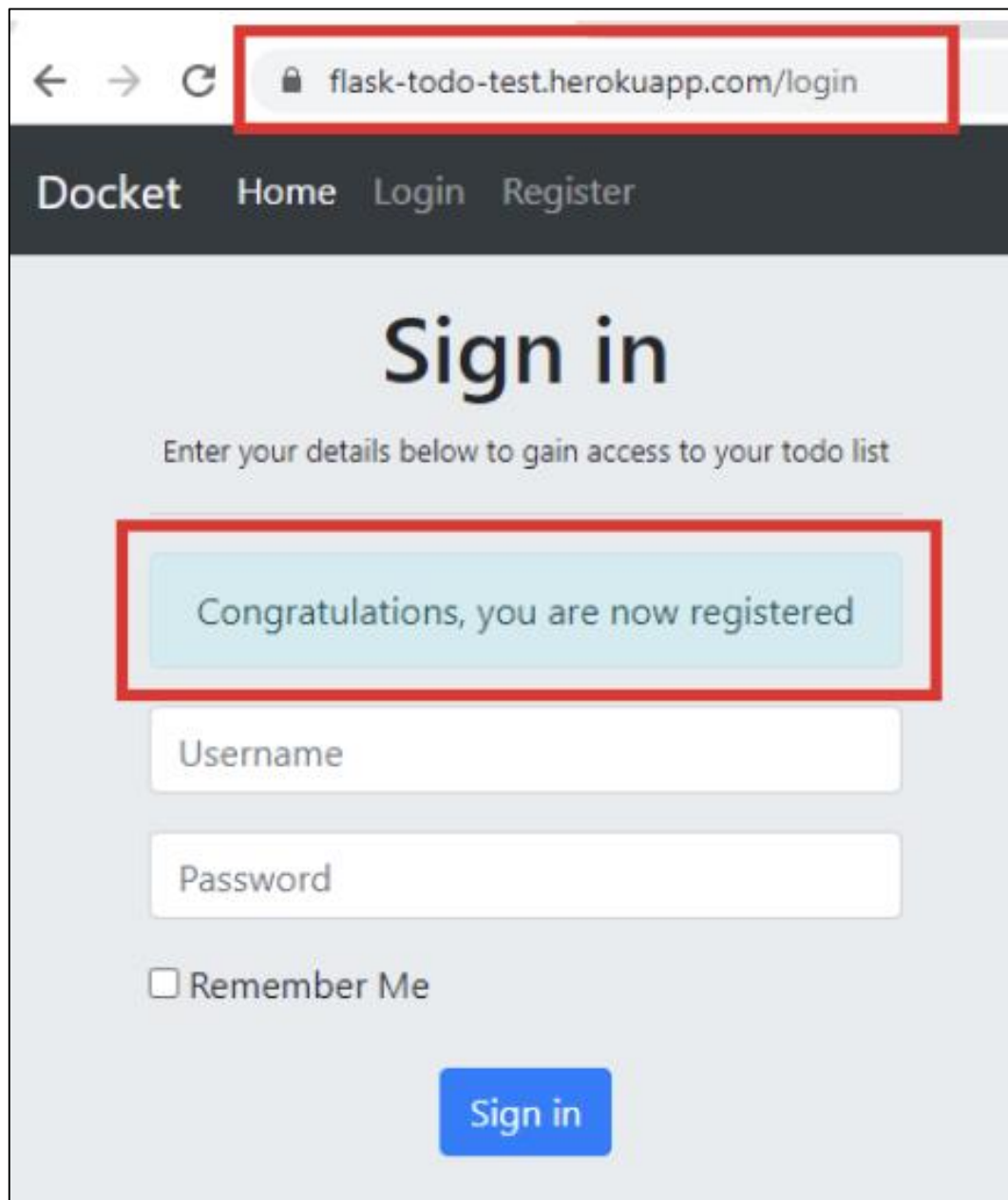
Ждем загрузку страницы корзины

```
wait.until(EC.url_contains("cart"))
```

Проверяем наличие названия товара в корзине

```
cart_item = driver.find_element(By.CLASS_NAME, "inventory_item_name")
```

```
assert "Sauce Labs Backpack" in cart_item.text, "Товар не найден в корзине"
```



Пример необходимости нескольких проверок в тесте:

При тестировании регистрации нужно проверить:

- ✓ появление надписи об успешной регистрации и
- ✓ переход пользователя на страницу авторизации

Пример: используем встроенный метод assert в Python

```
# confirm registration has been successful
```

```
# check if congratulations message contains the correct text
```

```
message = driver.find_element(By.XPATH, "/html[1]/body[1]/div.....").text
```

```
assert message == "Well done, You've Registered "
```

Первая проверка

```
# check user is routed to login page
```

```
current_url = driver.current_url
```

```
assert current_url == https://docket-test.herokuapp.com/register
```

Вторая проверка

```
driver.quit()
```

```
> assert message == "Well done, You've Registered"
E       assert 'Congratulati...ow registered' == "Well done, You've Registered"
E         - Well done, You've Registered
E         + Congratulations, you are now registered

tests\test_register.py:35: AssertionError
===== short test summary info =====
FAILED tests/test_register.py::test_register_user - assert 'Congratulati...ow reg
"
===== 1 failed test in 0.01s =====

(pytest_check-IMHGYL0F) C:\Users\ryanh\Desktop\pytest_check>
```

По результатам видно, что вторая проверка url даже не проводилась, тест упал уже на первом assert

Проблема

Проблема встроенного метода `assert` в Python заключается в том, что он останавливает выполнение теста при первой ошибке. В некоторых сценариях в этом и вправду есть смысл. То есть нет смысла продолжать тест, если часть приложения не работает. Однако бывает и такое, что есть смысл провести все проверки независимо от того, выпадет ли ошибка на одной из них или нет.

Такой подход часто используется при тестировании веб-сервисов. При проверке тела ответа нужно проверить каждое возвращаемое значение, а затем вывести, какие из них были не такими, как ожидалось, вместо того чтобы останавливать тест при ошибке в первом `assert`.

Решение: плагин `Pytest-check`

`Pytest-check` – плагин `Pytest`, который позволяет обернуть все `assert`-ы теста в один результат `pass/fail`. Например, если у вас есть 3 `assert`-а и первый из них выдал `fail`, то `Pytest-check` продолжит выполнять оставшиеся 2. Затем он сообщит о том, что тест провалился, если одна или несколько проверок придут с результатом `fail`.

<https://habr.com/ru/companies/otus/articles/533552/>

Пример: используем метод assert из Pytest-check

```
import pytest_check as check
```

```
.....
```

```
# confirm registration has been successful
```

```
# check if congratulations message contains the correct text
```

```
message = driver.find_element(By.XPATH, "/html[1]/body[1]/div.....").text  
check.equal(message, "Congratulations, you are now registered")
```

Первая проверка

```
# check user is routed to login page
```

```
current_url = driver.current_url
```

```
check.is_in("login", current_url)
```

Вторая проверка

```
driver.quit()
```

Теперь видно, что fail вернулся в двух проверках.

Можно рассматривать это как один провалившийся тест, но теперь можно конкретно узнать, какие несколько проверок вернули fail.

```
===== FAILURES =====  
test_register_user  
FAILURE:  
assert 'Congratulati...ow registered' == 'Congratulati...w registered!'  
- Congratulations, you are now registered!  
?  
+ Congratulations, you are now registered  
tests\test_register.py:35 in test_register_user() -> check.equal(message, "Cong  
FAILURE:  
assert '1' in 'https://docket-test.herokuapp.com/login'  
tests\test_register.py:39 in test_register_user() -> check.is_in("1", current_u  
-----  
Failed Checks: 2
```

Отчёты

Для генерации отчетов на Java используют **TestNG** (Testing Next Generation), фреймворк для автоматизации тестирования на языке Java, используемый для Unit, функциональных и интеграционных тестов.

Для Python используют Pytest с плагином pytest-html.

Для C# используют ExtentReports или встроенную отчетность NUnit.